

HR Table (5 Columns)

Steps to Do:

1. **Create table** with 5 columns → EmpID, EmpName, Department, Salary, JoiningDate.
2. **Insert at least 3 records** (employees with different departments).
3. **Select all records** using SELECT * FROM HR;.
4. **Select specific columns** (only EmpName, Salary).
5. **Filter records** (e.g., employees with Salary > 50000).

output

The screenshot displays a SQL File 3 window with the following queries:

```
1 • create database task1
2 • use task1;
3 • create table hr(EmpID int primary key,
4 •     EmpName varchar(30) not null,
5 •     Department varchar(50) not null,
6 •     Salary decimal(10,2) not null,
7 •     JoiningDate varchar(30));
8 • select * from hr;
9 • insert into hr(EmpID,EmpName,Department,Salary,JoiningDate)
10 values(1,"sravani","IT",9000,"10-09-2025"),(2,"kiranmy","IT",71000,"10-10-2025"),(3,"devika","IT",2000,"10-11-2025");
11 • select * from hr;
12 • select EmpName,Salary from hr;
13 • select * from hr
```

The Result Grid shows the following data:

EmpID	EmpName	Department	Salary	JoiningDate
1	sravani	IT	12000.00	10-09-2025
2	kiranmy	IT	11000.00	10-10-2025
3	devika	IT	13000.00	10-11-2025
NULL	NULL	NULL	NULL	NULL

The Output window shows the following actions and messages:

#	Time	Action	Message
17	16:13:00	select *from hr Salary > 50000	Error Code: 1064. You have an em
18	16:13:45	select *from hr where Salary > 50000 LIMIT 0, 1000	0 row(s) returned
19	16:14:18	select EmpName,Salary from hr LIMIT 0, 1000	3 row(s) returned
20	16:14:18	select * from hr LIMIT 0, 1000	3 row(s) returned
21	16:14:42	select EmpName,Salary from hr LIMIT 0, 1000	3 row(s) returned
22	16:14:42	select * from hr LIMIT 0, 1000	3 row(s) returned

DairyMilk Table (7 Columns)

Steps to Do:

1. **Create table** with 7 columns → ProductID, ProductName, Weight_grams, Price, Flavour, MfgDate, ExpDate.
2. **Insert at least 3 products** (different flavors & weights).
3. **Select all records** using SELECT * FROM DairyMilk;.

4. **Select specific columns** (only ProductName, Price, Flavour).
5. **Filter products** where Price > 100.
6. **Filter products** where Flavour = 'Oreo'.

output

The screenshot shows a SQL IDE window titled 'SQL File 3*' with a toolbar and a 'Limit to 1000 rows' dropdown. The script contains the following SQL statements:

```

6  Price decimal(10,2) not null,
7  Flavour varchar(30) not null,
8  MfgDate varchar(30),
9  ExpDate varchar(30));
10 select *
11 from DairyMilk;
12 insert into DairyMilk(ProductID,ProductName,Weight_grams,Price,Flavour,MfgDate,ExpDate)
13 values(10,"nuts",89.9,100,"chocolate","10-09-2020","10-07-2023"),
14 (11,"bubbles",99.9,150,"honey","10-09-2021","10-07-2024"),
15 (12,"darymilk",88.9,1000,"oreo","10-09-2023","1-07-2025");
16 select ProductName,Price,Flavour from DairyMilk;
17 select Flavour,Weight_grams from DairyMilk;
18 • select * from DairyMilk
19 where Price > 100;
20 • select * from DairyMilk
21 where Flavour = 'Oreo';

```

Below the script is the 'Result Grid' showing the results of the queries. The first query (line 12) returns a single row with all columns. The second query (line 16) returns three rows with ProductName, Price, and Flavour. The third query (line 17) returns three rows with Flavour and Weight_grams. The fourth query (line 18) returns three rows with all columns, filtered by Price > 100. The fifth query (line 20) returns three rows with all columns, filtered by Flavour = 'Oreo'.

ProductID	ProductName	Weight_grams	Price	Flavour	MfgDate	ExpDate
12	darymilk	88.90	1000.00	oreo	10-09-2023	1-07-2025
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Below the result grid is the 'Output' window showing the execution log. It displays three successful queries with their execution times and the number of rows returned.

#	Time	Action
9	16:48:05	select Flavour,Weight_grams from DairyMilk LIMIT 0, 1000
10	16:48:52	select * from DairyMilk where Price > 100 LIMIT 0, 1000
11	16:49:03	select * from DairyMilk where Flavour = 'Oreo' LIMIT 0, 1000

RoyalEnfield Table (9 Columns)

Steps to Do:

1. **Create table** with 9 columns → BikeID, ModelName, EngineCC, Price, Color, Mileage_KMPL, LaunchYear, ABS, FuelType.

2. **Insert at least 3 bike models** with different specs.
3. **Select all records** using `SELECT * FROM RoyalEnfield;`
4. **Select specific columns** (only `ModelName`, `Price`, `Mileage_KMPL`).
5. **Filter bikes** where `Price > 1,90,000`.
6. **Filter bikes** launched after 2020.
7. **Filter bikes** with `ABS = 1`.

output

The screenshot shows a SQL IDE window with a script titled "SQL File 3* x". The script contains the following SQL statements:

```

8  Mileage_KMPL decimal(30) not null,
9  LaunchYear int not null,
10 ABS bit not null,
11 FuelType varchar(20));
12 • select * from RoyalEnfield;
13 • insert into RoyalEnfield(BikeID,ModelName,EngineCC,Price,Color,Mileage_KMPL,LaunchYear,ABS,FuelType)
14 values(111, "Classic", 349, 185000, "Black", 35.00, 2019, 1, "Petrol"),
15 (112, "Meteor", 349, 210000, "Red", 32.00, 2021, 1, "Petrol"),
16 (113, "Himalayan", 411, 225000, "White", 30.00, 2022, 1, "Petrol");
17 • select * from RoyalEnfield;
18 • select ModelName,Price,Mileage_KMPL from RoyalEnfield;
19 • select * from RoyalEnfield
20 where Price > 190000;
21 • select * from RoyalEnfield
22 where LaunchYear>2020;
23 • select * from RoyalEnfield
24 where ABS = 1;

```

Below the script, the "Result Grid" shows the data for the `RoyalEnfield` table:

	BikeID	ModelName	EngineCC	Price	Color	Mileage_KMPL	LaunchYear	ABS	FuelType
▶	111	Classic	349	185000.00	Black	35.00	2019	1	Petrol
	112	Meteor	349	210000.00	Red	32.00	2021	1	Petrol
	113	Himalayan	411	225000.00	White	30.00	2022	1	Petrol
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Below the result grid, the "Output" section shows the execution of the queries:

#	Time	Action	Message
✓ 17	17:16:29	select * from RoyalEnfield where Price > 190000 LIMIT 0, 1000	2 row(s) ret
✓ 18	17:17:59	select * from RoyalEnfield where LaunchYear>2020 LIMIT 0, 1000	2 row(s) ret
✓ 19	17:18:58	select * from RoyalEnfield where ABS = 1 LIMIT 0, 1000	3 row(s) ret